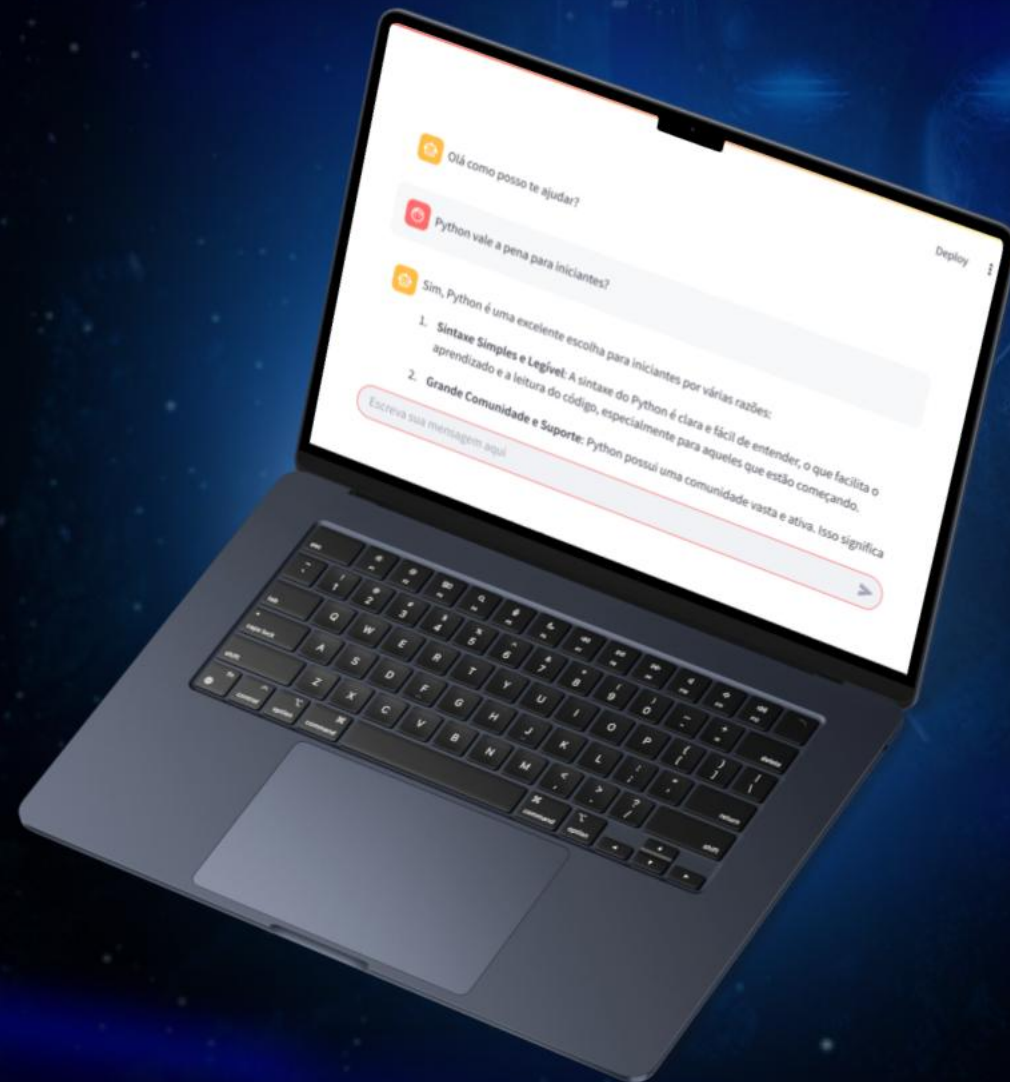




AULA 04

CHATBOT COM IA EM TEMPO REAL



Parte 1

Introdução

O que Vamos Aprender?

Nesta aula, você vai trabalhar em um **Projeto Python Dev**, criando sites e sistemas utilizando Python e desenvolvendo um **chatbot com Inteligência Artificial**.

Durante o projeto, você vai compreender o que é um **framework**, entender a diferença entre **front-end** e **back-end**, e aprender a integrar soluções de IA para criar um chatbot capaz de responder perguntas dentro de um cenário específico, como o contexto de uma empresa.

A proposta é ensinar a construir seu próprio “ChatGPT”, ou seja, um sistema que gera respostas automatizadas e mantém contexto em conversas.

Antes de começar, é recomendado assistir à aula completa para entender toda a lógica do projeto, e depois realizar os testes utilizando o material disponibilizado, como a apostila e a gravação.

Nesta aula, **não serão utilizados arquivos prévios**. O projeto será desenvolvido a partir de um editor de código em branco, sem base de dados ou estrutura anterior. Todo o sistema será construído do zero.

Esta é uma das aulas mais importantes da jornada, reunindo diversos conceitos fundamentais em um único projeto prático.

ChatBot com IA



Pensei em aprender Python, vale a pena?



Aprender Python pode ser uma decisão muito vantajosa, dependendo dos seus interesses e objetivos de carreira. Aqui estão algumas razões pelas quais vale a pena considerar aprender Python:

1. **Popularidade e Comunidade:** Python é uma das linguagens de programação mais populares do mundo, com uma grande comunidade de usuários. Isso significa que há muitos recursos, tutoriais e bibliotecas disponíveis para ajudá-lo a aprender e resolver problemas.
2. **Versatilidade:** Python é uma linguagem extremamente versátil e pode ser usada em diversas áreas, como desenvolvimento web, automação, análise de dados, aprendizado de máquina, ciência de dados, desenvolvimento de scripts e prototipagem rápida.
3. **Fácil de Aprender:** A sintaxe do Python é simples e legível, o que o torna uma boa escolha para iniciantes em programação. A curva de aprendizado é relativamente suave comparada a outras linguagens.
4. **Demanda de Mercado:** Há uma alta demanda por programadores que conhecem Python, especialmente em campos como ciência de dados, inteligência artificial e desenvolvimento

Escreva sua mensagem aqui

Configuração Inicial e Estrutura do Projeto

Abra o **Visual Studio Code**.

Acesse o menu **Arquivo** → **Abrir Pasta** e escolha ou crie uma pasta para armazenar o projeto.

Crie um arquivo Python dentro da pasta. No exemplo, use `main.py`, mas qualquer nome que termine com `.py` funciona.

Todo o projeto será desenvolvido **em um único arquivo**, facilitando a compreensão da lógica completa do sistema.

Definindo o Passo a Passo do Sistema

Antes de escrever o código, defina o **passo a passo do sistema**.

Em automações ou análises de dados, o passo a passo costuma ser linear: primeiro uma etapa, depois outra.

Em um site ou chatbot, o passo a passo representa:

- O que existirá dentro do sistema.
- Como o usuário vai interagir com esses elementos.
- Como o sistema deve responder.

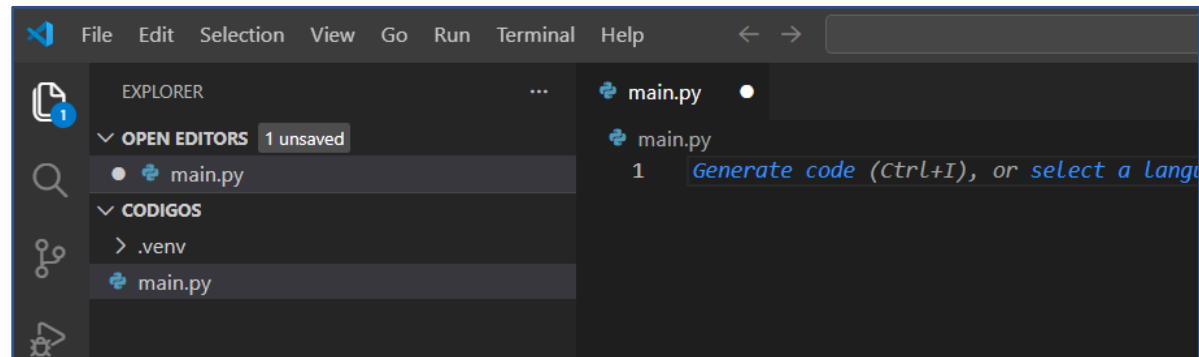
Planejar o chatbot significa estruturar o funcionamento da aplicação, não apenas organizar comandos em ordem linear. Este planejamento ajuda a organizar seu raciocínio e evita confusões durante a implementação.

Estruturando o Funcionamento do Chat

O planejamento não segue um modelo linear tradicional. Aqui, você precisa definir claramente:

- O que o sistema precisa ter.
- Como ele deverá funcionar.

Para visualizar isso, execute o projeto final novamente e identifique os principais elementos que precisarão ser construídos.



Configuração Inicial e Estrutura do Projeto

Elementos do Sistema

Título do sistema: visível na parte superior da página.

Campo de mensagem (input do chat): espaço onde o usuário digita perguntas.

O input funciona como a **porta de entrada** da interação entre o usuário e o chatbot.

Funcionamento do Chat

A lógica do sistema se baseia na repetição de um mesmo fluxo sempre que você envia uma mensagem.

Ao digitar uma pergunta, como:

“Vale a pena estudar na vida?”

“Quem construiu as pirâmides do Egito?”

O sistema executa três etapas principais:

- **Exiba** a mensagem enviada na tela, construindo visualmente o histórico da conversa.

- **Envie** a mensagem para a Inteligência Artificial, que processará a pergunta e gerará a resposta.
- **Mostre** a resposta retornada pela IA no chat.

Esse ciclo se repete continuamente, permitindo que cada nova mensagem seja incorporada ao histórico, criando uma experiência interativa e contextualizada.

```
main.py
main.py
1  # título
2  # input do chat (campo de mensagem)
3  # cada mensagem que o usuário enviar:
4      # mostrar a mensagem que o usuário enviou no chat
5      # pegar a pergunta e enviar para uma IA responder
6      # exibir a resposta da IA na tela
7
8  # Streamlit -> apenas com Python criar o frontend e o backend
9  # a IA que vamos usar: OpenAI
10
```


Escolhendo a Ferramenta para Construir o Sistema

Para criar um site, chatbot ou qualquer sistema web em Python, você precisa de uma ferramenta específica, chamada **framework**.

O que são Frameworks

Frameworks são pacotes de código criados para resolver desafios específicos. A diferença para bibliotecas comuns é que frameworks possuem **regras e estruturas próprias** que devem ser seguidas para o projeto funcionar corretamente. Além de funcionalidades prontas, eles definem uma forma organizada de desenvolver a aplicação.

Principais Frameworks para Criar Sites em Python

Entre as ferramentas mais utilizadas estão:

- **Streamlit**
- **Flask**
- **Django**
- **FastAPI**

Cada framework possui características próprias e regras de funcionamento específicas.

Flask: O Flask exige que você organize manualmente a estrutura do projeto, oferecendo liberdade de configuração.

Django: O Django cria automaticamente arquivos e estruturas quando o projeto é iniciado, fornecendo uma base mais robusta e estruturada.

Streamlit: O Streamlit possui uma estrutura dinâmica e executa todo o código sempre que há uma atualização na aplicação, tornando o desenvolvimento direto e simples para este tipo de projeto.

Por que Utilizar Streamlit

Nesta aula, a ferramenta escolhida é o **Streamlit**, pois permite criar um chatbot com interface visual de forma mais direta. Ele possibilita que **front-end e back-end** sejam desenvolvidos utilizando apenas Python, sem precisar de outras linguagens.

Escolhendo a Ferramenta para Construir o Sistema

Front-end

O front-end corresponde ao que o usuário vê na tela, incluindo:

- O título do sistema
- Os textos exibidos
- O campo de mensagem
- As respostas apresentadas

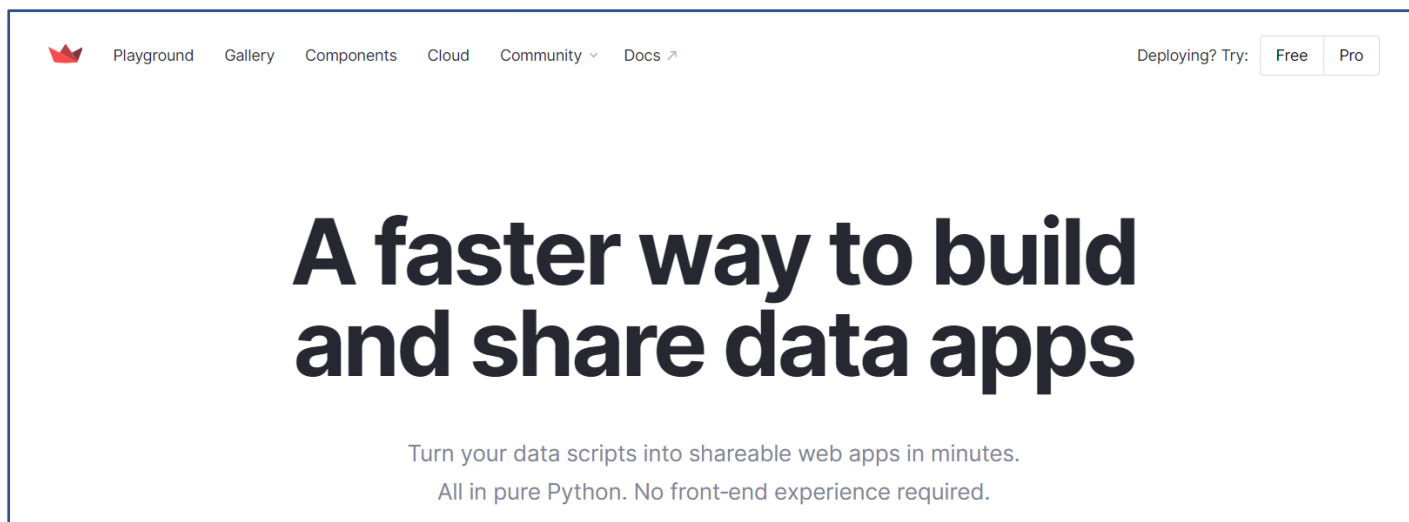
Trata-se de toda a interface visual da aplicação.

Back-end

O back-end corresponde à lógica por trás da interface, envolvendo:

- Receber a mensagem do usuário
- Enviá-la para a IA
- Processar a resposta
- Exibir o resultado

O Streamlit permite desenvolver front-end e back-end de forma integrada, utilizando **um único código em Python**.



Parte 2

Bibliotecas Necessárias

Bibliotecas Necessárias

Instalação das Bibliotecas

Instalação da Ferramenta

O primeiro passo é instalar o framework escolhido e a Inteligência Artificial utilizada para gerar as respostas do chatbot.

Inteligência Artificial Utilizada

Nesta aula, a IA escolhida é a **OpenAI**, pois é conhecida, possui integração direta e oferece créditos gratuitos para testes durante o aprendizado.

Instalação das Bibliotecas

Nós vamos precisar instalar as seguintes bibliotecas:

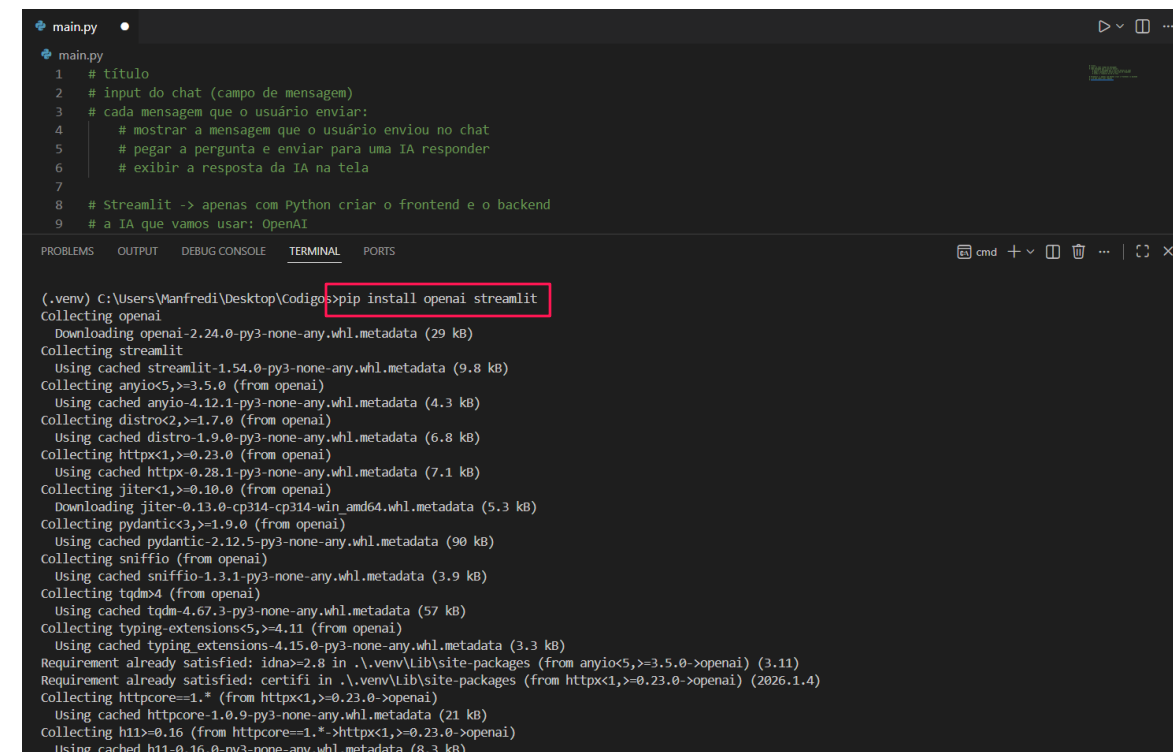
- **Streamlit** - # responsável pela interface web - <https://docs.streamlit.io/>
- **OpenAI** - # responsável pela comunicação com a IA - <https://docs.streamlit.io/>

pip install openai streamlit

Você precisa das bibliotecas instaladas para que consiga fazer esse projeto. Caso contrário vai ter algumas mensagens de

erro ao executar o seu código. Então é importante que faça a instalação antes de iniciar o projeto.

Se tiver dúvidas ou quiser se aprofundar, você pode consultar a documentação oficial das bibliotecas. A documentação é sempre uma excelente fonte de consulta para entender parâmetros, funções disponíveis e boas práticas de utilização.



```
main.py
main.py
1 # título
2 # input do chat (campo de mensagem)
3 # cada mensagem que o usuário enviar:
4 # mostrar a mensagem que o usuário enviou no chat
5 # pegar a pergunta e enviar para uma IA responder
6 # exibir a resposta da IA na tela
7
8 # Streamlit -> apenas com Python criar o frontend e o backend
9 # a IA que vamos usar: OpenAI

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) C:\Users\Manfredi\Desktop\Codigos>pip install openai streamlit
Collecting openai
  Downloading openai-2.24.0-py3-none-any.whl.metadata (29 kB)
Collecting streamlit
  Using cached streamlit-1.54.0-py3-none-any.whl.metadata (9.8 kB)
Collecting anyio<5,>=3.5.0 (from openai)
  Using cached anyio-4.12.1-py3-none-any.whl.metadata (4.3 kB)
Collecting distro<2,>=1.7.0 (from openai)
  Using cached distro-1.9.0-py3-none-any.whl.metadata (6.8 kB)
Collecting httpx<1,>=0.23.0 (from openai)
  Using cached httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting jiter<1,>=0.10.0 (from openai)
  Downloading jiter-0.13.0-cp314-cp314-win_amd64.whl.metadata (5.3 kB)
Collecting pydantic<3,>=1.9.0 (from openai)
  Using cached pydantic-2.12.5-py3-none-any.whl.metadata (90 kB)
Collecting sniffio (from openai)
  Using cached sniffio-1.3.1-py3-none-any.whl.metadata (3.9 kB)
Collecting tqdm>4 (from openai)
  Using cached tqdm-4.67.3-py3-none-any.whl.metadata (57 kB)
Collecting typing-extensions<5,>=4.11 (from openai)
  Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Requirement already satisfied: idna>=2.8 in .\venv\Lib\site-packages (from anyio<5,>=3.5.0->openai) (3.11)
Requirement already satisfied: certifi in .\venv\Lib\site-packages (from httpx<1,>=0.23.0->openai) (2026.1.4)
Collecting httpcore==1.* (from httpx<1,>=0.23.0->openai)
  Using cached httpcore-1.0.9-py3-none-any.whl.metadata (21 kB)
Collecting h11>=0.16 (from httpcore==1.*->httpx<1,>=0.23.0->openai)
  Using cached h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
```

Parte 3

Código em Python

Construção do Sistema

Importando o Streamlit

Com as bibliotecas instaladas, abra o arquivo Python criado anteriormente e comece o desenvolvimento.

Como o projeto utiliza o **Streamlit**, o primeiro passo é importar a biblioteca:

```
import streamlit as st
```

Por que utilizar as st?

O “as st” define um **apelido** para o Streamlit. Sempre que precisar usar alguma funcionalidade da biblioteca, basta escrever st. ao invés de streamlit.. Esse padrão é amplamente utilizado e aparece na documentação oficial.

Criando Elementos no Sistema

O Streamlit funciona de forma direta: para criar qualquer elemento na interface, utilize:

```
st.algo_que_quero_criar()
```

Alguns exemplos:

st.write() → escreve texto na tela

st.button() → cria um botão

st.chat_input() → cria um campo de chat

st. seguido de diferentes tipos de gráficos (barra, área, linha)

A lógica é sempre: st. + elemento desejado.

Criando o Primeiro Elemento

Inserindo o Título do Chatbot

O primeiro elemento será o título do sistema. Utilize:

```
st.write("Chatbot com IA")
```

O texto pode ser alterado para qualquer nome desejado, como:

“Chatbot com IA”

“Hashtag Chatbot”

“Chatbot do Lira”

Com essa linha, você cria um site simples contendo um título.

Construção do Sistema

```
main.py
main.py
1  # título
2  # input do chat (campo de mensagem)
3  # cada mensagem que o usuário enviar:
4      # mostrar a mensagem que o usuário enviou no chat
5      # pegar a pergunta e enviar para uma IA responder
6      # exibir a resposta da IA na tela
7
8  # Streamlit -> apenas com Python criar o frontend e o backend
9  # a IA que vamos usar: OpenAI
10 # pip install openai streamlit
11
12 import streamlit as st
13
14 st.write("Chatbot com IA")
```

Executando o Sistema

Como Rodar um Projeto com Streamlit

Não utilize o botão de execução padrão do editor, pois o Python executa o arquivo e encerra imediatamente, sem manter o site ativo.

O **Streamlit** precisa ser executado de forma específica para que o sistema fique disponível e interativo.

Executando pelo Terminal

Abra o terminal.

Certifique-se de que não há processos do Streamlit rodando, interrompendo qualquer execução anterior e feche o que estiver ativo.

No terminal, digite: ***streamlit run main.py***

Onde:

streamlit → nome da ferramenta

run → indica que o projeto deve ser executado

main.py → nome do arquivo com o código (substitua se tiver outro nome)

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
• PS C:\Users\Manfredi\Desktop\Codigos> & C:/Users/Manfredi/Desktop/Codigos/.venv/Scripts/Activate.ps1
• (.venv) PS C:\Users\Manfredi\Desktop\Codigos> & c:\Users\Manfredi\Desktop\Codigos\.venv\Scripts\python.exe main.py
• (.venv) PS C:\Users\Manfredi\Desktop\Codigos> streamlit run main.py

You can now view your Streamlit app in your browser.

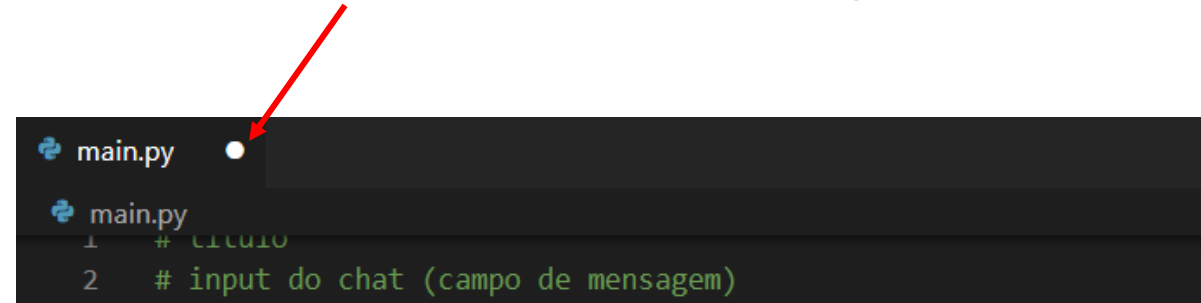
Local URL: http://localhost:8501
Network URL: http://192.168.15.11:8501
```

Construção do Sistema

Atenção: O Arquivo Precisa Estar Salvo

Antes de executar, garanta que o arquivo esteja salvo.

No Visual Studio Code, alterações não salvas são indicadas por uma bolinha branca ao lado do nome do arquivo.



Pressione **Ctrl + S** ou acesse **Arquivo** → **Save** para salvar.

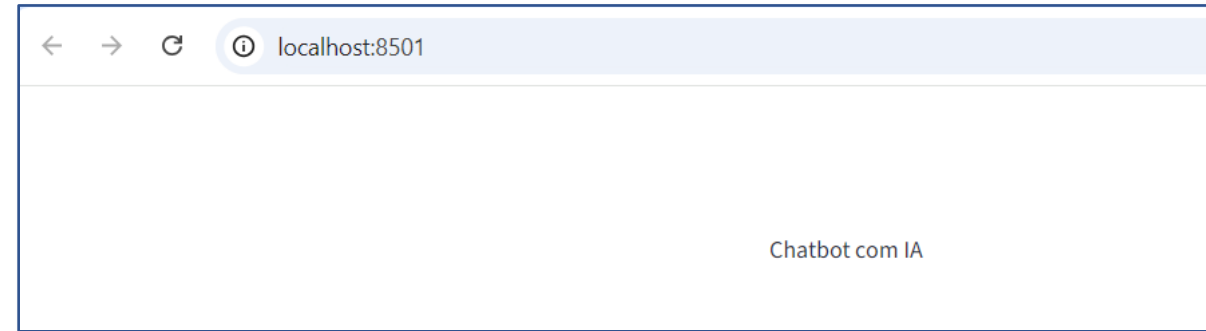
Após salvar, execute o comando `streamlit run main.py`.

O Que Acontece ao Executar

Após pressionar **Enter**, o Streamlit executa o projeto.

Uma nova aba se abre no navegador, exibindo o site com o conteúdo definido no código.

O site roda localmente (`localhost`) e os links exibidos no terminal podem ser usados para acessar a página.



Atualizando o Site

Enquanto o processo estiver ativo no terminal, o site permanece rodando.

Se fizer alterações no código, como:

`st.write("Lira ensinando Python para a galera")`

Siga estes passos:

Salve o arquivo (**Ctrl + S**)

Vá até o navegador

Atualize a página (**F5** ou botão **Rerun**)

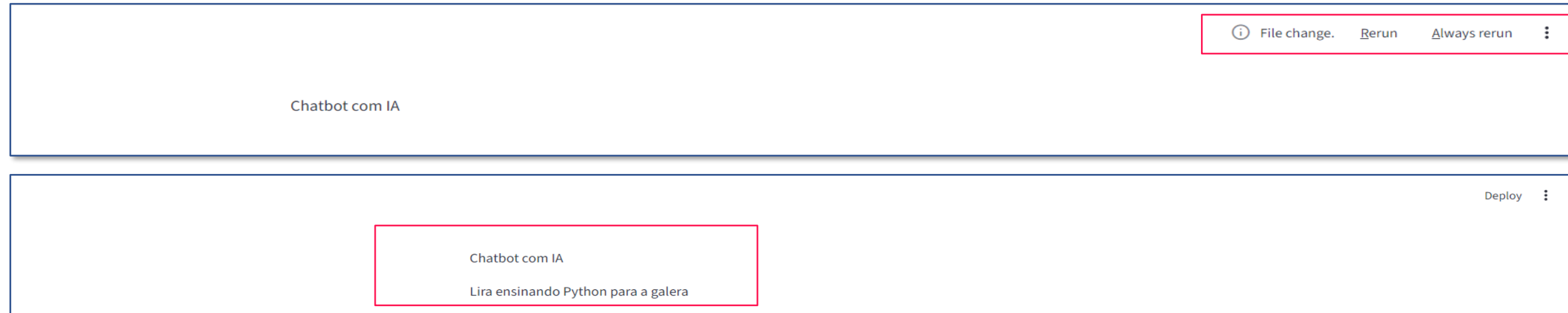
O Streamlit identifica alterações e exibe a mensagem:

Source file changed

Código em Python

Construção do Sistema

Você pode atualizar manualmente ou clicar no botão de atualização.



Interrompendo a Execução

Para parar o servidor do Streamlit:

Clique na lixeira do terminal ou pressione **Ctrl + C** no terminal

Quando interrompido, o site sai do ar e o navegador exibirá falha de conexão.

Para colocar o site no ar novamente, execute:

`streamlit run main.py`

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
streamlit + v [icon] [icon] ... |

• PS C:\Users\Manfredi\Desktop\Codigos> & C:/Users/Manfredi/Desktop/Codigos/.venv/Scripts/Activate.ps1
• (.venv) PS C:\Users\Manfredi\Desktop\Codigos> & c:\Users\Manfredi\Desktop\Codigos\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Manfredi\Desktop\Codigos> streamlit run main.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.15.11:8501

[icon]
```


Construção do Sistema

Recapitulando o Funcionamento

O processo de trabalho com Streamlit segue sempre a mesma lógica:

- Edite o código.
- Salve o arquivo.
- Atualize o navegador.

Se o servidor não estiver rodando, execute:

streamlit run nome_do_arquivo.py

O terminal pode permanecer minimizado enquanto o site continua funcionando normalmente.

Como o Streamlit Funciona

O Streamlit Cria o Código do Site Sozinho

O Streamlit gera automaticamente toda a estrutura interna do site, incluindo HTML, CSS e JavaScript.

Você escreve apenas em Python, e o framework constrói a interface web nos bastidores.

Salvamento Automático

O Visual Studio Code permite configurar **Auto Save**, mas durante o projeto a prática será:

- Manter o terminal rodando e realizar alterações no código
- Salvar manualmente (Ctrl + S)
- Atualizar o navegador (F5)

Personalizando o Título

O Streamlit utiliza formato **Markdown** para textos.

→ título grande

→ título menor

→ título ainda menor

Exemplo:

```
st.write("# Chatbot com IA")
```

Ao salvar e atualizar a página, o texto aparece maior. A hashtag não aparece visualmente, apenas define o estilo do texto.

Construção do Sistema

Criando o Campo de Mensagem

Inserindo o Chat Input

Crie o campo onde o usuário digita mensagens:

```
texto_usuario = st.chat_input("Digite sua mensagem")
```

O que é st.chat_input()?

A função st.chat_input() cria o campo de entrada do chat, semelhante ao WhatsApp.

Dentro dos parênteses, você define um **placeholder**, que é o texto exibido antes do usuário digitar algo.

Exemplo: "Digite sua mensagem"

Após salvar e atualizar o site, o campo aparece na parte inferior da página.

Chatbot com IA

Digite sua mensagem



Capturando a Mensagem do Usuário

Quando o usuário digita uma mensagem e pressiona Enter, o texto digitado é armazenado na variável `texto_usuario`, podendo ser utilizado no código posteriormente.

Testando com Print

Para verificar se a mensagem está sendo capturada corretamente:

```
print(texto_usuario)
```

Se não houver mensagem enviada, o terminal exibe None.

Se uma mensagem for enviada, por exemplo "Bora aprender Python", o terminal exibirá exatamente esse texto.

```
11
12  import streamlit as st
13
14  st.write("# Chatbot com IA") # markdown
15
16  texto_usuario = st.chat_input("Digite sua mensagem")
17
18  print(texto_usuario)
```

Código em Python

Construção do Sistema

```
(.venv) C:\Users\Manfredi\Desktop\Codigos>streamlit run main.py
```

```
You can now view your Streamlit app in your browser.
```

```
Local URL: http://localhost:8501
```

```
Network URL: http://192.168.15.8:8501
```

```
None
```

```
Bora aprender Python
```

```
█
```

Como o Streamlit Executa o Código

O Streamlit executa o código inteiro sempre que há interação:

- Ao enviar uma mensagem
- Ao clicar em um botão
- Ao atualizar a página

Durante a execução:

- O título é recriado
- O campo de chat é recriado
- A mensagem enviada é armazenada
- O print() exibe o valor no terminal

O Problema do None

Se nenhuma mensagem for enviada, `texto_usuario` fica vazio e o terminal exibe None.

Isso ocorre porque o código imprime a variável mesmo quando ela não contém texto.

Tratando o Problema do None

O valor None indica que a variável não possui nenhum valor.

Para exibir a mensagem apenas quando o usuário realmente enviar algo, crie uma condição:

if texto_usuario:

print(texto_usuario)

Se houver texto digitado → o conteúdo será exibido

Se não houver texto → nenhuma ação será executada

Tudo que estiver indentado dentro do if será executado apenas quando a condição for verdadeira.

Código em Python

Construção do Sistema

```
main.py X
main.py > ...
12 import streamlit as st
13
14 st.write("# Chatbot com IA") # markdown
15
16 texto_usuario = st.chat_input("Digite sua mensagem")
17
18 if texto_usuario:
19     print(texto_usuario)
```

Chatbot com IA

Lira começou de novo a enviar mensagem



```
None
Bora aprender Python
None
None
Lira começou de novo a enviar mensagem
█
```

Estrutura da Lógica do Chatbot

O bloco `if texto_usuario:` será o ponto central da lógica do chatbot.

Dentro dele, implemente:

- Exibir a mensagem do usuário no chat
- Enviar a pergunta para a IA
- Exibir a resposta da IA

Compatibilidade e Personalização da Interface

O Streamlit possui **design responsivo**, adaptando a interface a diferentes tamanhos de tela

É possível personalizar cor de fundo, cor dos textos, espaçamentos e outros ajustes visuais

Nesta etapa, será utilizado o **padrão visual da ferramenta**, que já oferece interface organizada

Todo o desenvolvimento será feito apenas em Python; o Streamlit gera automaticamente a estrutura interna da interface.

Construção do Sistema

Vantagens de Criar um Chatbot Próprio

Ao criar seu próprio chatbot, você poderá:

- Integrar a IA ao contexto específico de um negócio
- Controlar a interface e experiência do usuário
- Automatizar fluxos personalizados
- Incorporar a IA dentro de um sistema próprio

Principais vantagens:

- Integrar a inteligência da OpenAI dentro do próprio site
- Utilizar o chatbot para atendimento ao cliente
- Fornecer informações específicas da empresa para as respostas

Exibindo Mensagens no Chat

Para exibir a mensagem diretamente na interface do chat, utilize:

```
mensagem = st.chat_message("user") mensagem.write(texto_usuario)
```

"Nome" → exibe a letra inicial do nome como identificação

"user" → ícone de usuário

"assistant" → ícone de robô

Funcionamento:

Cria-se uma mensagem identificada como "user"

.write() exibe o texto dentro da mensagem

Ao salvar e atualizar, a mensagem digitada aparece no chat.

```
main.py X
main.py > ...
12 import streamlit as st
13
14 st.write("# Chatbot com IA") # markdown
15
16 texto_usuario = st.chat_input("Digite sua mensagem")
17
18 if texto_usuario:
19     print(texto_usuario)
20     st.chat_message("user").write(texto_usuario)
21     # Nome
22     # user
23     # assistant
```

Construção do Sistema

Chatbot com IA



Bora aprender Python

Digite sua mensagem



Simulando a Resposta da IA

Antes da integração com a IA real, teste a lógica com uma resposta simulada:

```
resposta_ia = "Você perguntou: " + texto_usuario
```

Apenas repete a pergunta enviada

Não há processamento inteligente

Permite validar o fluxo do chat

Exemplo:

Usuário digita: "Bora aprender Python?"

Resposta exibida: "Você perguntou: Bora aprender Python?"

```
14 st.write("# Chatbot com IA", # markdown)
15
16 texto_usuario = st.chat_input("Digite sua mensagem")
17
18 if texto_usuario:
19     print(texto_usuario)
20     st.chat_message("user").write(texto_usuario)
21     # Nome
22     # user
23     # assistant
24
25     # ia respondeu
26     resposta_ia = "Você perguntou: " + texto_usuario
27
28     st.chat_message("assistant").write(resposta_ia)
```

Chatbot com IA



Bora aprender Python



Você perguntou: Bora aprender Python

Construção do Sistema

Exibindo a Resposta da IA

Para exibir a resposta simulada:

```
mensagem_ia = st.chat_message("assistant")
mensagem_ia.write(resposta_ia)
```

Fluxo completo:

- Usuário envia a mensagem
- Mensagem é exibida como "user"
- Resposta simulada é criada
- Resposta é exibida como "assistant"

Com isso, a estrutura básica do chat está concluída:

- Usuário envia mensagem
- Sistema responde

Limitações Atuais

Atualmente, o sistema possui algumas limitações:

Resposta da IA não é inteligente

- Resposta apenas repete o texto do usuário
- Próximo passo: integrar a OpenAI para receber respostas dinâmicas

Mensagens não estão sendo armazenadas

- Cada nova mensagem apaga a anterior
- O histórico não é mantido porque o Streamlit executa o código inteiro a cada interação

Chatbot com IA



Bora aprender Python



Você perguntou: Bora aprender Python

Chatbot com IA



Sim, perguntei mesmo



Você perguntou: Sim, perguntei mesmo

Construção do Sistema

Resolvendo o Problema do Histórico

Para armazenar mensagens, dois conceitos do Python serão utilizados:

- Lista
- Dicionário

Conceito 1: Lista no Python

Uma lista é uma coleção de informações

```
nomes = ["Lira", "Thalia", "Guilherme", "Alon"]
```

Adicionar item: `.append()`

```
nomes.append("Michelle")
```

Acessar item: pelo índice (começa em 0)

```
nomes[0] # retorna "Lira"
```

```
nomes[2] # retorna "Guilherme"
```

Para o chat: cada nova mensagem será adicionada a uma lista para preservar o histórico

```
1
2 # listas
3 lista_nova = []
4 nomes = ["Lira", "Thalia", "Guilherme", "Alon"]
5 print(nomes)
6
7 nomes.append("Michely") # adiciona informações na lista
8 print(nomes)
9
10 # dicionarios
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS + v

```
.exe c:/Users/Manfredi/Desktop/Codigos/auxiliar.py
['Lira', 'Thalia', 'Guilherme', 'Alon']
['Lira', 'Thalia', 'Guilherme', 'Alon', 'Michely']
```

```
1
2 # listas
3 lista_nova = []
4 nomes = ["Lira", "Thalia", "Guilherme", "Alon"]
5 print(nomes[0]) # acessar o primeiro item da lista
6
7 nomes.append("Michely") # adiciona informações na lista
8 print(nomes)
9
10 # dicionarios
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS + v

Open file in editor (ctrl + click)

```
(.venv) C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/Desktop/Codigos/.venv/Scripts/python
.exe c:/Users/Manfredi/Desktop/Codigos/auxiliar.py
Lira
['Lira', 'Thalia', 'Guilherme', 'Alon', 'Michely']
```

Construção do Sistema

Conceito 2: Dicionário no Python

Um dicionário armazena pares chave → valor

`Idades = {"Lira": 31, "Thalia": 23, "Alon": 45}`

Acessar valor: pela chave

`idades["Lira"]` # retorna 31

Adicionar ou editar valor:

`idades["Michelle"] = 25` # adiciona

`idades["Lira"] = 26` # edita

```
auxiliar.py > ...
10  # dicionario
11
12  idades = {"Lira": 31, "Thalia": 23, "Alon": 45}
13  print(idades["Lira"]) # pega informação no dicionário -> dicionario[chave]
14  idades["Michely"] = 25 # adicionando uma informação
15  idades["Lira"] = 26 # editando uma informação
16  print(idades)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
(.venv) C:\Users\Manfredi\Desktop\Codigos>C:/Users/Manfredi/Desktop/Codigos/.venv/Script
fredi/Desktop/Codigos/auxiliar.py
Lira
['Lira', 'Thalia', 'Guilherme', 'Alon', 'Michely']
31
{'Lira': 26, 'Thalia': 23, 'Alon': 45, 'Michely': 25}
```

Construção do Sistema

Aplicando ao Chat

Cada mensagem precisa de duas informações:

- Quem enviou (role)
- O conteúdo da mensagem (content)

Exemplo de mensagem:

```
mensagem1 = {"role": "assistant", "content": "Bora aprender Python?"}
```

```
mensagem2 = {"role": "user", "content": "Bora aprender."}
```

```
mensagem3 = {"role": "assistant", "content": "Então vamos começar a aula."}
```

O Chat é uma Lista de Mensagens

Todo o chat é uma lista de mensagens:

```
lista_mensagens = [mensagem1, mensagem2, mensagem3]
```

Estrutura final:

```
[{'role': 'assistant', 'content': 'Bora aprender Python?'}, {'role': 'user', 'content': 'Bora aprender.'}, {'role': 'assistant', 'content': 'Então vamos começar a aula.'}]
```

Essa é a estrutura padrão utilizada por todos os sistemas de IA.

Cada mensagem possui:

- role → quem enviou
- content → conteúdo da mensagem

No Streamlit, quando se usa:

- `st.chat_message("user")`
- `st.chat_message("assistant")`

Está sendo definido o role.

Adicionando Nova Mensagem na Lista

```
nova_mensagem = {"role": "user", "content": "Opa agora bora botar Python pro jogo"}
```

```
lista_mensagens.append(nova_mensagem)
```

`.append()` adiciona a mensagem no final da lista

Código em Python

Construção do Sistema

```
19
20 # role = quem é o usuário -> user ou assistant
21 # content = conteúdo da mensagem
22 mensagem1 = {"role": "assistant", "content": "Bora aprender Python"}
23 mensagem2 = {"role": "user", "content": "Bora sim, bora aprender"}
24 mensagem3 = {"role": "assistant", "content": "Então vamos começar a aula"}
25
26 lista_mensagens = [mensagem1, mensagem2, mensagem3]
27
28 nova_mensagem = {"role": "user", "content": "Opa, agora bora botar Python pro jogo"}
29 lista_mensagens.append(nova_mensagem)
30
31 print(lista_mensagens)
```

O chat = lista de dicionários → cada dicionário = uma mensagem → cada mensagem possui role e content

Por Que Explicar Lista e Dicionário?

As IAs esperam receber uma lista de mensagens nesse formato

Fluxo ao enviar uma pergunta:

- Criar dicionário com a nova mensagem
- Adicionar à lista de mensagens
- Enviar lista completa para a IA
- IA retorna dicionário com a resposta
- Resposta adicionada à lista

Estrutura usada por qualquer IA conversacional:

```
[{"role": "...", "content": "..."}, {"role": "...", "content": "..."}]
```

Implementando a Lógica no Código Real

Criar a lista de mensagens no código principal:

```
lista_mensagens = []
```

Problema: Streamlit roda o código inteiro a cada mensagem, então a lista seria recriada vazia

Solução: `st.session_state`

`session_state` = armazenamento da sessão do usuário

Lógica:

- Se informação existe → mantém
- Se não → cria

Criando lista apenas se não existir:

```
If not "lista_mensagens" in st.session_state:
```

```
    st.session_state["lista_mensagens"] = []
```

Construção do Sistema

Agora:

- `st.session_state["lista_mensagens"]` = lista oficial do chat
- Não será apagada a cada execução

Criando e Adicionando Mensagens na Lista

Sempre que nova mensagem é enviada:

- Criar dicionário da mensagem
- Adicionar na lista

Mensagem do Usuário

`mensagem_usuario = {"role": "user", "content": texto_usuario}`

"role": "user" → indica usuário

"content": `texto_usuario` → texto digitado

Mensagem da IA

`mensagem_ia = {"role": "assistant", "content": resposta_ia}`

"role": "assistant" → indica IA

"content": `resposta_ia` → texto da resposta

Adicionando na Lista

`st.session_state["lista_mensagens"].append(mensagem_usuario)`

`st.session_state["lista_mensagens"].append(mensagem_ia)`

Agora, toda mensagem enviada é armazenada corretamente

```
main.py > ...
12 import streamlit as st
13
14 st.write("# Chatbot com IA") # markdown
15
16 if not "lista_mensagens" in st.session_state:
17     st.session_state["lista_mensagens"] = []
18
19 texto_usuario = st.chat_input("Digite sua mensagem")
20
21 if texto_usuario:
22     print(texto_usuario)
23     st.chat_message("user").write(texto_usuario)
24     mensagem_usuario = {"role": "user", "content": texto_usuario}
25     st.session_state["lista_mensagens"].append(mensagem_usuario)
26     # Nome
27     # user
28     # assistant
29
30     # ia respondeu
31     resposta_ia = "Você perguntou: " + texto_usuario
32
33     st.chat_message("assistant").write(resposta_ia)
34     mensagem_ia = {"role": "assistant", "content": resposta_ia}
35     st.session_state["lista_mensagens"].append(mensagem_ia)
36
37 print(st.session_state["lista_mensagens"])
```

Construção do Sistema

Testando se Está Funcionando

Para verificar se a lista está sendo atualizada, foi feito um print no final do código:

```
print(st.session_state["lista_mensagens"])
```

Ao atualizar a página:

- A lista começa vazia

Ao enviar a primeira mensagem, duas mensagens aparecem:

- Mensagem do usuário
- Resposta da IA

Exemplo da estrutura exibida:

```
[{"role": "user", "content": "Qual é a galera? Bora aprender Python"}, {"role": "assistant", "content": "Você perguntou: Qual é a galera? Bora aprender Python"}]
```

Ao enviar outra mensagem:

- A lista continua crescendo

Agora terá:

- Primeira mensagem do usuário
- Primeira resposta da IA
- Segunda mensagem do usuário
- Segunda resposta da IA

Conclusão: a lista está 100% funcional

```
25     st.session_state["lista_mensagens"].append(mensagem_usuario)
26     # Nome
27     # user
28     # assistant
29
30     # ia respondeu
31     resposta_ia = "Você perguntou: " + texto_usuario
32
33     st.chat_message("assistant").write(resposta_ia)
34     mensagem_ia = {"role": "assistant", "content": resposta_ia}
35     st.session_state["lista_mensagens"].append(mensagem_ia)
36
37     print(st.session_state["lista_mensagens"])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS +

```
[{'role': 'user', 'content': 'Coe galera, bora aprender Python?'}, {'role': 'assistant', 'content': 'Você perguntou: Coe galera, bora aprender Python?'}, {'role': 'user', 'content': 'nova mensagem'}, {'role': 'assistant', 'content': 'Você perguntou: nova mensagem'}]
```

Construção do Sistema

Chatbot com IA



nova mensagem



Você perguntou: nova mensagem

Digite sua mensagem



O Problema Atual

Visualmente, a interface ainda exibe apenas:

- Última mensagem do usuário
- Última resposta da IA

O armazenamento está correto, mas a exibição **não**

O Próximo Desafio

Percorrer toda a lista de mensagens e exibir cada uma delas

Para cada mensagem:

- Identificar o role
- Exibir o content correspondente

Será feito usando um **loop**

Como Funcionam os Cookies do `session_state`?

"lista_mensagens" entre aspas → chave do dicionário do `session_state`

`session_state` funciona como um dicionário armazenado na sessão

O Que São Cookies?

Informações armazenadas no navegador sobre a sessão

Exemplos:

- Pesquisas armazenadas
- ID do usuário
- Preferências registradas

Ficam salvas enquanto a sessão estiver ativa

Construção do Sistema

Conteúdo dos Cookies

Essencialmente um dicionário: {"id_usuario": 123456, "email": "email@exemplo.com", "preferencias": {...}}

Cada site tem seus próprios cookies → não são compartilhados

Percorrendo a Lista de Mensagens

Agora começa a parte em que tudo o que foi ensinado sobre listas e dicionários se encaixa. A ideia é simples: para cada mensagem dentro da lista de mensagens, exibir essa mensagem na tela.

O loop é feito assim:

```
for mensagem in st.session_state["lista_mensagens"]:
```

Isso significa que, para cada mensagem que existe na lista armazenada em `st.session_state`, será executado o bloco de código dentro do loop.

Cada mensagem é um dicionário com a estrutura: {"role": "...", "content": "..."}.

Para acessar as informações, usamos `mensagem["role"]` para saber quem enviou e `mensagem["content"]` para pegar o conteúdo.

```
main.py > ...
12 import streamlit as st
13
14 st.write("# Chatbot com IA") # markdown
15
16 if not "lista_mensagens" in st.session_state:
17     st.session_state["lista_mensagens"] = []
18
19 texto_usuario = st.chat_input("Digite sua mensagem")
20
21 for mensagem in st.session_state["lista_mensagens"]:
22     role = mensagem["role"]
23     content = mensagem["content"]
24     st.chat_message(role).write(content)
25
26 if texto_usuario:
27     print(texto_usuario)
28     st.chat_message("user").write(texto_usuario)
29     mensagem_usuario = {"role": "user", "content": texto_usuario}
30     st.session_state["lista_mensagens"].append(mensagem_usuario)
31     # Nome
32     # user
33     # assistant
34
35     # ia respondeu
36     resposta_ia = "Você perguntou: " + texto_usuario
37
38     st.chat_message("assistant").write(resposta_ia)
39     mensagem_ia = {"role": "assistant", "content": resposta_ia}
40     st.session_state["lista_mensagens"].append(mensagem_ia)
41
42 print(st.session_state["lista_mensagens"])
```


Construção do Sistema

Conteúdo dos Cookies

Essencialmente um **dicionário**: {"id_usuario": 123456, "email": "email@exemplo.com", "preferencias": {...}}

Cada site tem seus próprios cookies → não são compartilhados

Percorrendo a Lista de Mensagens

Agora começa a parte em que tudo o que foi ensinado sobre listas e dicionários se encaixa. A ideia é simples: para cada mensagem dentro da lista de mensagens, exibir essa mensagem na tela.

O loop é feito assim:

```
for mensagem in st.session_state["lista_mensagens"]:
```

Isso significa que, para cada mensagem que existe na lista armazenada em `st.session_state`, será executado o bloco de código dentro do loop.

Cada mensagem é um dicionário com a estrutura: {"role": "...", "content": "..."}.

Para acessar as informações, usamos `mensagem["role"]` para saber quem enviou e `mensagem["content"]` para pegar o conteúdo.

Exibindo a Mensagem na Tela

O responsável por mostrar a mensagem na interface é o `st.chat_message()`. Dentro do loop, a lógica completa fica assim:

```
for mensagem in st.session_state["lista_mensagens"]: with  
st.chat_message(mensagem["role"]):  
st.write(mensagem["content"])
```

Para cada mensagem, o código identifica quem enviou, cria o bloco visual na tela e escreve o conteúdo da mensagem.

O Resultado

Quando a página é atualizada, a lista começa vazia. Ao enviar uma mensagem, ela é adicionada à lista, a resposta da IA também é incluída, e o loop percorre a lista inteira, exibindo todas as mensagens novamente. Agora não aparece apenas a última mensagem; o histórico completo é mantido e o chat funciona como um chat real.

O `print` usado anteriormente para depuração não é mais necessário, porque todas as mensagens já estão visíveis na interface.

Código em Python

Construção do Sistema

```
main.py > ...
12 import streamlit as st
13
14 st.write("# Chatbot com IA") # markdown
15
16 if not "lista_mensagens" in st.session_state:
17     st.session_state["lista_mensagens"] = []
18
19 texto_usuario = st.chat_input("Digite sua mensagem")
20
21 for mensagem in st.session_state["lista_mensagens"]:
22     role = mensagem["role"]
23     content = mensagem["content"]
24     st.chat_message(role).write(content)
25
26 if texto_usuario:
27     print(texto_usuario)
28     st.chat_message("user").write(texto_usuario)
29     mensagem_usuario = {"role": "user", "content": texto_usuario}
30     st.session_state["lista_mensagens"].append(mensagem_usuario)
31     # Nome
32     # user
33     # assistant
34
35     # ia respondeu
36     resposta_ia = "Você perguntou: " + texto_usuario
37
38     st.chat_message("assistant").write(resposta_ia)
39     mensagem_ia = {"role": "assistant", "content": resposta_ia}
40     st.session_state["lista_mensagens"].append(mensagem_ia)
41
42 print(st.session_state["lista_mensagens"])
```



Bora aprender Python?



Você perguntou: Bora aprender Python?



Sim, perguntei



Você perguntou: Sim, perguntei



ih a la, agora foouooooi



Você perguntou: ih a la, agora foouooooi



caramba



Você perguntou: caramba

Digite sua mensagem



Construção do Sistema

O Que Falta Agora

A estrutura do chat está pronta: a lista persiste, as mensagens são adicionadas corretamente e o histórico é exibido. O que ainda não está inteligente é a resposta da IA, que até agora é apenas um texto padrão. O próximo passo será enviar a lista de mensagens para a inteligência artificial (ChatGPT) e receber respostas reais baseadas no contexto.

Sobre Treinar a IA

Na aula de hoje, não é necessário treinar a IA. Será utilizado um modelo já pronto da OpenAI (ChatGPT), e é possível escolher qual modelo usar, de acordo com o que estiver disponível na API. Quando a integração com a OpenAI for realizada, a própria API retorna a lista de modelos disponíveis.

Criar uma IA Própria Sem OpenAI

É possível, mas criar um modelo do zero exige muito trabalho. O caminho mais comum é baixar um modelo pronto e ajustá-lo ou treiná-lo para a necessidade específica. Esse processo envolve aprender como baixar, ajustar e treinar modelos, o que não faz parte do escopo atual.

O Que É Uma API e Como Funciona

API significa **Application Programming Interface**, ou seja, uma interface de programação que permite se comunicar com um sistema. Simplificando, é a forma oficial que uma empresa disponibiliza para você interagir com o sistema dela. Na prática, a empresa define: “eu espero que você me mande as informações nesse formato”. Se forem enviadas de outro jeito, não funcionam.

Padrões de Estrutura

Atualmente, existem basicamente dois padrões para organizar mensagens de chat: a lista de dicionários e o padrão do LangChain. Mesmo o LangChain, internamente, utiliza uma lista de dicionários. No fim das contas, esse é o padrão dominante e mais utilizado por qualquer IA conversacional.

Integrando com a OpenAI

A biblioteca já foi instalada e a importação é feita assim:

```
from openai import OpenAI
```

É importante notar a diferença: openai em minúsculo é o pacote Python, enquanto OpenAI com maiúscula é a classe usada para criar a conexão com a API.

Código em Python

Construção do Sistema

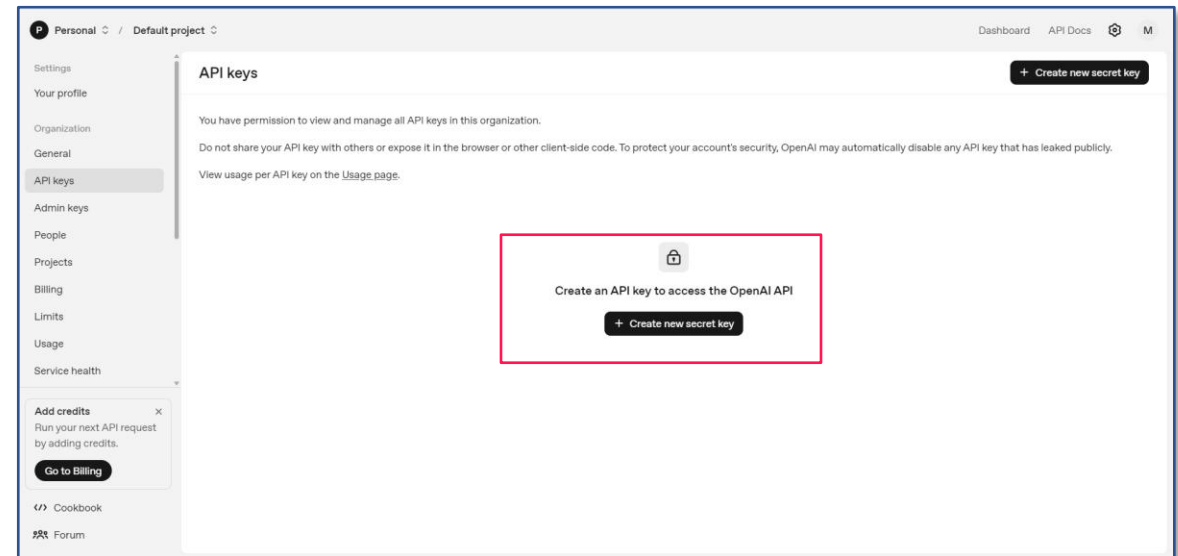
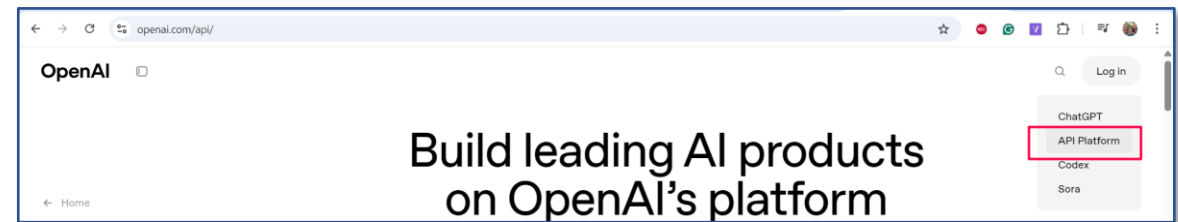
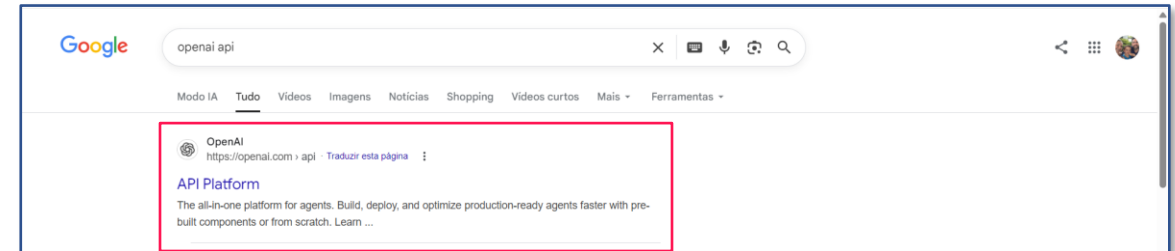
Para usar a API, é necessário criar um cliente:

```
Modelo_ia = OpenAI(api_key="SUA_CHAVE_AQUI")
```

Essa chave de API funciona como uma senha, identificando quem está fazendo a requisição, quem será cobrado e qual conta está associada ao uso.

Para obtê-la, você deve abrir o navegador, procurar pela OpenAI API, entrar no site oficial, acessar a plataforma da API e fazer login (a criação de conta é gratuita). Depois de logado, vá em **Settings**, clique em **API Keys**, crie uma nova chave, dê um nome para ela e copie a chave gerada. Essa chave aparece apenas uma vez, então é preciso guardá-la com cuidado e colá-la no código.

Criar a conta é gratuito, e geralmente a OpenAI oferece créditos iniciais para testes. Depois que os créditos forem usados, é possível adicionar valores pequenos para continuar utilizando a API. Para testes e aprendizado, os créditos gratuitos geralmente são suficientes. É fundamental não compartilhar a chave, cada pessoa deve criar a sua e é possível ter várias chaves diferentes.



Construção do Sistema

Create new secret key

Owned by

You

Service account

This API key is tied to your user and can make requests against the selected project. If you are removed from the organization or project, this key will be disabled.

Name Optional

chaveaulajornada

Project

Default project

Permissions

All

Restricted

Read only

Cancel

Create secret key

Save your key

Please save your secret key in a safe place since **you won't be able to view it again**. Keep it secure, as anyone with your API key can make requests on your behalf. If you do lose it, you'll need to generate a new one.

[Learn more about API key best practices](#)

~~AIzaSyDj5T...CQ...I24T...JL...M...C...I7...~~ Copy

Permissions

Read and write API resources

Done

Construção do Sistema

Começando a Integração com a IA

Antes, a resposta da IA era apenas um texto fixo. Agora, a resposta passará a vir do modelo da OpenAI. É importante separar a resposta completa da API do texto que será exibido na interface.

A API retorna não apenas o texto, mas também informações sobre tokens usados, metadados e outros dados técnicos. Por isso, criamos dois objetos: `resposta_ia` para guardar tudo e `texto_resposta_ia` apenas com o conteúdo que será mostrado no chat.

Para criar a resposta, usamos o modelo de chat da OpenAI, já que estamos desenvolvendo um chatbot. A chamada é feita assim:

```
resposta_ia = modelo_ia.chat.completions.create(
    messages=st.session_state["lista_mensagens"],
    model="gpt-4o")
```

Dentro do `.create()` enviamos duas informações obrigatórias: o modelo a ser usado e a lista completa de mensagens do chat (`st.session_state["lista_mensagens"]`). Isso garante que o modelo responda com base no contexto completo da conversa.

```
38     # ia respondeu
39     resposta_ia = modelo_ia.chat.completions.create(
40         messages=st.session_state["lista_mensagens"],
41         model="gpt-4o"
42     )
43     print(resposta_ia)
44     texto_resposta_ia = "Você perguntou: " + texto_usuario
45
46     st.chat_message("assistant").write(texto_resposta_ia)
47     mensagem_ia = {"role": "assistant", "content": texto_resposta_ia}
48     st.session_state["lista_mensagens"].append(mensagem_ia)
49
50     print(st.session_state["lista_mensagens"])
```

```
{'role': 'assistant', 'content': 'Você perguntou: ih a la, agora fooooooi'}, {'role': 'assistant', 'content': 'Você perguntou: caramba'}]
ChatCompletion(id='chatcmpl-CyRkbi4do9wVpboy9dxiaDpkwzAud', choices=[Choice(finish_reason='stop', index=0, logprobs=None, message=ChatCompletionMessage(content='Sim, aprender Python é uma ótima ideia por várias razões:\n\n1. **Facilidade de Aprendizagem**: Python é conhecido por ser fácil de aprender e legível, o que o torna uma ótima linguagem para iniciantes.\n\n2. **Versatilidade**: Python é uma linguagem de propósito geral que pode ser usada em diversas áreas, como desenvolvimento de software, análise de dados, inteligência artificial, automação, desenvolvimento de aplicativos e muito mais.\n\n3. **Comunidade Ativa**: Python possui uma comunidade global grande e ativa. Isso significa que há uma abundância de recursos de aprendizado, como tutoriais, documentação e fóruns de suporte, onde você pode encontrar ajuda quando surgirem dúvidas.\n\n4. **Bibliotecas e Frameworks**: Python tem uma extensa coleção de bibliotecas e frameworks que facilitam o desenvolvimento de projetos complexos. Por exemplo, Django e Flask para desenvolvimento web, Pandas e NumPy para análise de dados, e TensorFlow e PyTorch para aprendizado de máquina.\n\n5. **Demanda do Mercado**: Python é amplamente utilizado nas indústrias de tecnologia e ciência de dados, o que cria uma alta demanda por desenvolvedores de Python, especialmente em campos relacionados à IA e machine learning.'), logprobs=None)])
```


Construção do Sistema

É possível escolher qualquer modelo disponível na OpenAI, como GPT-4, GPT-4o ou modelos mais novos. No caso, foi escolhido o "gpt-4o" por ser mais barato, funcionar bem para testes e servir como um bom modelo inicial para desenvolvimento. Modelos mais novos costumam ser mais caros e poderosos, então o ideal é começar simples e evoluir conforme necessário.

Como Funciona a Requisição

Quando o usuário envia uma mensagem, o código envia a lista de mensagens para a OpenAI. A API processa, o sistema aguarda alguns segundos e a resposta retorna. Agora existe comunicação externa com a API.

O objeto `resposta_ia` contém muitas informações: ID da requisição, modelo usado, data, tokens utilizados, estatísticas e metadados, além de uma lista de respostas possíveis dentro do campo `choices`. Para pegar o texto da resposta, acessamos o primeiro item da lista, e dentro dele usamos `message.content`.

`texto_resposta_ia = resposta_ia.choices[0].message.content`

Assim, `resposta_ia` contém tudo, e `texto_resposta_ia` contém apenas o texto que será exibido no chat.

```
37
38     # ia respondeu
39     resposta_ia = modelo_ia.chat.completions.create(
40         messages=st.session_state["lista_mensagens"],
41         model="gpt-4o"
42     )
43     texto_resposta_ia = resposta_ia.choices[0].message.content
44
45     st.chat_message("assistant").write(texto_resposta_ia)
46     mensagem_ia = {"role": "assistant", "content": texto_resposta_ia}
47     st.session_state["lista_mensagens"].append(mensagem_ia)
48
49     print(st.session_state["lista_mensagens"])
```

Chatbot com IA



Python, vale a pena aprender?



Com certeza, aprender Python vale muito a pena. Python é uma das linguagens de programação mais populares e versáteis atualmente e tem uma série de vantagens que fazem dela uma excelente escolha para iniciantes e profissionais experientes. Aqui estão algumas razões para aprender Python:

1. **Facilidade de Aprendizado:** Python possui uma sintaxe clara e concisa, o que facilita o

Construção do Sistema

Fluxo Final da Resposta

Quando o usuário pergunta:

Python vale a pena aprender?

O sistema:

- Envia o histórico completo.
- Recebe a resposta completa da API.
- Extrai apenas o texto.
- Exibe o texto na tela.
- Adiciona o texto na lista de mensagens.
- Agora o chatbot realmente responde com inteligência.
- Não é mais texto fixo.
- É uma resposta gerada com base no contexto.

Resultado

Pergunta enviada:

- Python vale a pena aprender?

Resposta recebida:

- Com certeza, aprender Python vale muito a pena...

O chatbot passa a funcionar como um sistema real de conversa com IA.

Estrutura funcionando:

- Persistência ✓
- Histórico ✓
- Integração com OpenAI ✓
- Resposta inteligente ✓

Agora o projeto deixa de ser um teste local e começa a virar uma aplicação real.

A partir daqui começam as evoluções mais avançadas.

Parte 4

Conclusão

Conclusão

Nesse projeto tivemos desafios diferentes dos anteriores. Aqui não foi apenas criar uma aplicação visual. Nós construímos um **chatbot completo integrado com Inteligência Artificial**.

Você aprendeu a estruturar mensagens com **listas e dicionários**, entendeu como funciona o **st.session_state** para manter histórico, integrou com a **API da OpenAI**, fez **requisições reais** para um modelo de IA e tratou a **resposta retornada pela API**. Ou seja, você não apenas utilizou IA. Você entendeu como ela funciona dentro de um sistema.

Também vimos que a IA não é perfeita. Ela pode errar, pode interpretar algo errado ou responder de forma imprecisa. E entender isso é importante, porque programar não é esperar perfeição da ferramenta — é saber **controlar, validar e estruturar** o que está sendo feito.

Você viu que é possível limitar o comportamento usando **System Prompt**, tratar mensagens antes de enviar, **controlar custos**, entender como funciona o **histórico** e até como preparar o sistema para um **deploy online**.

Um ponto muito importante continua sendo a **prática**. Não é sobre decorar código. É sobre entender a lógica, testar, errar, ajustar, melhorar e repetir. É assim que você começa a ganhar **autonomia** para criar seus próprios projetos.

Não precisa começar pelo mais complexo. Começando pelo simples — como fizemos aqui — você vai juntando vários blocos de conhecimento até construir algo **robusto**.

Agora você já consegue criar um **chatbot com IA funcionando de verdade**.

A partir daqui, o próximo passo não é copiar código. É começar a **adaptar**, testar novas ideias, integrar com **banco de dados**, criar **regras mais avançadas** e evoluir o projeto.

Espero que tenham gostado dessa jornada e que continuem praticando, estudando e principalmente **construindo**.

Espero que tenham gostado da **Jornada Python** e espero que continuem praticando e estudando Python!

{JORNADA} PYTHON

100% GRATUITO E ONLINE

Ainda não segue a gente no Instagram e nem é inscrito no nosso canal do Youtube? Então corre lá!



@hashtagprogramacao



youtube.com/hashtag-programacao

